Algorithms **Spring Semester, 2026**

Homework 3 (Graded)

Professor: Dr. Mudassir Shabbir *Deadline: Mar 16, 2026.*

Instructions. This is a graded homework. Submit your solutions as a single PDF. Show all work; correct answers without justification will receive no credit. You may collaborate on high-level ideas but must write up solutions independently.

1. **[30 points]** *Union-Find models dynamic connectivity, which arises in network routing, image segmentation, and Kruskal's MST algorithm.*

   Recall the **Union-Find** data structure with two optimizations:

   - **Union by rank:** always attach the root of the lower-rank tree under the root of the higher-rank tree; if ranks are equal, choose either root and increment its rank by 1.
   - **Path compression:** during `Find`(x), make every node on the path from x to the root point directly to the root.

   Start with $n = 8$ elements $\{1, 2, 3, 4, 5, 6, 7, 8\}$, each initially its own component with rank 0.

   (a) **[12 pts]** Perform the following operations in order using union by rank. After *each* Union, draw the resulting forest. For every node shown, label its **rank** and indicate the **parent pointer** (or mark it as a root).

   $$\texttt{Union}(1,2), \quad \texttt{Union}(3,4), \quad \texttt{Union}(5,6), \quad \texttt{Union}(7,8)$$

   $$\texttt{Union}(1,3), \quad \texttt{Union}(5,7)$$

   $$\texttt{Union}(1,5)$$

   (b) **[12 pts]** Now perform `Find`(8) with path compression. Write down the sequence of nodes visited while traversing parent pointers from 8 to the root. Show the updated parent array after path compression, and draw the resulting forest. How many distinct components exist after all operations? List every element in each component.

   (c) **[6 pts]** Suppose that instead of union by rank you used **naive union** (always attach the second argument's root under the first argument's root). Perform only the first four unions:

   $$\texttt{Union}(1,2), \quad \texttt{Union}(1,3), \quad \texttt{Union}(1,4), \quad \texttt{Union}(1,5)$$

   Draw the resulting tree. What is the height of this tree, and why is this bad for subsequent `Find` operations?

2. **[35 points]** *The running median is used in streaming data analysis, real-time signal processing, and financial tick data.*

   You receive integers one at a time in a stream. After each insertion you must report the **median** of all integers seen so far. (If the count is even, report the average of the two middle values; if odd, report the middle value.)

   (a) **[10 pts]** Describe a data structure using exactly **two heaps** — a max-heap H_{lo} storing the smaller half of the elements, and a min-heap H_{hi} storing the larger half — that supports `Insert` and `GetMedian` each in $O(\log n)$ time. Your answer must specify: (i) the **size/ordering invariant** between H_{lo} and H_{hi}; (ii) the **Insert** procedure including when and how to rebalance; and (iii) how **GetMedian** uses the heap roots to return the answer in $O(1)$.

   (b) **[5 pts]** Prove that your invariant is maintained after every `Insert`. You may assume H_{lo} and H_{hi} each support `Insert` and `ExtractMax`/`ExtractMin` in $O(\log n)$.

   (c) **[20 pts]** Trace your data structure on the following input stream:

   $$5, \quad 3, \quad 8, \quad 1, \quad 9, \quad 2, \quad 7$$

   After *each* insertion, fill in one row of the table below.

Inserted	H_{lo} (max-heap, top first)	H_{hi} (min-heap, top first)	Median
5			
3			
8			
1			
9			
2			
7			

3. **[35 points]** *Implementing a queue from two stacks is a classic application of amortized analysis.*

   Implement a **FIFO queue** using two stacks S_{in} and S_{out}, each supporting `Push` and `Pop` in $O(1)$ worst case, as follows:

   - `Enqueue`(x): push x onto S_{in}.
   - `Dequeue`: if S_{out} is non-empty, pop and return its top. Otherwise, move *all* elements from S_{in} to S_{out} (by repeated pop-then-push), then pop and return the top of S_{out}.

   (a) **[12 pts]** Trace the data structure on the following sequence of operations. After each operation write the contents of S_{in} and S_{out} (top element listed first) and the value returned (if any).

Operation	S_{in} (top first)	S_{out} (top first)	Returned
`Enqueue`(1)			—
`Enqueue`(2)			—
`Enqueue`(3)			—
`Dequeue`			
`Enqueue`(4)			—
`Dequeue`			
`Dequeue`			

   (b) **[23 pts]** First, explain why a single `Dequeue` on a queue holding n elements can cost $\Theta(n)$ in the worst case. Then use the **accounting method** to prove that any sequence of n `Enqueue` and `Dequeue` operations costs $O(n)$ total. Charge **3 credits** per `Enqueue` and **1 credit** per `Dequeue`, and maintain the **credit invariant**: every element currently in S_{in} holds exactly 2 banked credits.

   For each of the three cases below, show that the credits charged are sufficient to pay the actual cost *and* that the credit invariant is maintained afterwards. Then conclude that the credit balance never goes negative, so the total actual cost is at most the total credits charged, i.e. $O(n)$.

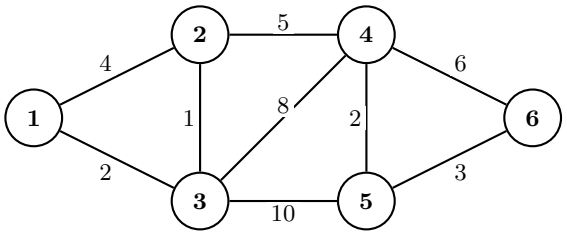
   (i) `Enqueue`(x): actual cost $= 1$. Where do the 3 charged credits go?

   (ii) `Dequeue` when S_{out} is **non-empty**: actual cost $= 1$. Show the 1 credit covers this; is the invariant on S_{in} affected?

   (iii) `Dequeue` when S_{out} is **empty** and S_{in} holds k elements: actual cost $= 2k + 1$. Show that the 1 charged credit plus the 2 banked credits per element exactly cover this cost.

4. **[30 points]** *Running Kruskal's algorithm by hand builds intuition for the cut property and reveals how sensitive the MST is to edge weight changes.*

   Consider the following weighted undirected graph G with vertices $\{1, 2, 3, 4, 5, 6\}$ and edges as shown.



(a) **[16 pts]** List all 9 edges in non-decreasing order of weight (break ties by smaller endpoint first), then trace Kruskal's algorithm. Fill in every row of the table. In the "Action" column write **Add** or **Skip**. In the "Reason" column, for a **Skip** name the cycle that would have been created; for an **Add** state which two components merge.

Step	Edge	Weight	Action	Reason / Components after
1				
2				
3				
4				
5				
6				
7				
8				
9				

(b) **[4 pts]** Draw the MST of G (label all edges and their weights) and state its total weight.

(c) **[10 pts]** When Kruskal's adds edge $(2, 4)$, it merges two components S and $V \setminus S$. Identify the cut $(S, V \setminus S)$ at that moment, list all edges crossing it with their weights, and verify using the **cut property** that $(2, 4)$ must belong to every MST of G.

Then suppose the weight of $(2, 4)$ is changed to w'. Find the exact threshold below which the MST is unchanged. For $w' = 9$, identify which edge replaces $(2, 4)$, draw the new MST, and state its total weight.